

Fiche de préparation à la soutenance PFA

Tastify – Plateforme intelligente de gestion de restaurant

Diouri Mehdi et El Kharrazi Ibtiha

Fiche personnelle de révision, orientée questions jury

Objectif de cette fiche

Lire ce document comme une carte mentale de soutenance : quoi dire au début, quoi montrer pendant la démo, comment défendre les choix techniques, et comment répondre sans paniquer aux questions IA, sécurité, tests, limites et perspectives.

1 Ouverture orale prête à dire

Pitch en 45 secondes

Bonjour, nous vous présentons **Tastify**, une plateforme web pour digitaliser la gestion d'un restaurant. L'idée est de réunir dans un seul système le portail client, le back-office du personnel, la prise de commande, le suivi cuisine, les réservations, les paiements simulés, la fidélité, le stock et l'analyse des avis clients. Notre valeur ajoutée est double : d'abord une gestion opérationnelle en temps réel, ensuite une couche d'intelligence applicative avec l'analyse de sentiment qui transforme les commentaires clients en indicateurs exploitables par le gérant.

Problématique

Dans beaucoup de restaurants, les commandes, les réservations, les paiements, les retours clients et le stock sont dispersés entre plusieurs outils ou gérés manuellement. Cela crée des pertes de temps, des erreurs, un manque de visibilité et une difficulté à exploiter les avis clients. Tastify propose une solution centralisée, testée et démontrable.

Transition vers la démo

Je vais d'abord montrer le parcours client, puis le back-office staff. Ensuite je montrerai le lien temps réel entre serveur et cuisine, le paiement par QR code, et enfin l'analyse de sentiment des avis qui alimente les statistiques.

2 Ce qu'il faut absolument retenir

Sujet	Réponse courte à donner
But du projet	Digitaliser le restaurant de bout en bout : client, staff, cuisine, stock, paiements, fidélité, avis et analytics.
Architecture	Backend Django REST commun, deux frontends React, MySQL, Redis, Celery, WebSockets et Docker Compose.
Pourquoi deux frontends ?	Les usages sont différents : interface simple côté client, outil opérationnel dense côté staff.
IA	Analyse de sentiment NLP sur les avis : Hugging Face si disponible, fallback local sinon.
Temps réel	WebSockets via Redis pour notifier le staff et synchroniser le KDS cuisine.
Sécurité	JWT, cookies de refresh, permissions DRF et contrôle des rôles côté API.
Tests	346 tests backend, 93% de couverture, 121 scénarios Playwright staff, 81 client.
Limite honnête	Paiement simulé et IA dépendante d'un service externe, avec fallback local.

3 Architecture à défendre

Vue simple de l'architecture

Client React	→	API Django REST	→	MySQL
Back-office React	→	Django Channels	→	Redis WebSocket
Avis client	→	Celery	→	Hugging Face / fallback local

Composant	Rôle dans Tastify
Django REST Framework	Expose les API métier : utilisateurs, menu, commandes, tables, réservations, paiements, avis, stock, fidélité.
React client	Portail public : menu, réservation, panier, paiement, compte, fidélité et avis.
React back-office	Interface staff : dashboard, salle, KDS, menu, stock, RH, avis, paramètres.
MySQL	Stockage relationnel des données métier.
Redis	Broker, cache opérationnel et support des WebSockets.
Celery	Traitements asynchrones : analyse de sentiment, tâches de fond, orchestration.
Docker Compose	Lancement reproductible de l'ensemble des services pour la soutenance et le développement.

Phrase à dire si on demande pourquoi Docker

Docker évite le problème du “ça marche sur ma machine”. Toute la stack démarre avec la même configuration : backend, base de données, Redis, Celery et les deux interfaces. Pour une soutenance, c'est un vrai gain de fiabilité.

4 Démo recommandée en 15 minutes

1. **Portail client** : accueil, menu, catégories, plats appréciés.
2. **Expliquer les recommandations** : elles utilisent les avis et les scores de sentiment.
3. **Back-office gérant** : dashboard, statistiques, avis clients, stock.
4. **Serveur** : plan de salle, choisir une table, ajouter des plats, envoyer en cuisine.
5. **Cuisinier** : KDS, passer un plat en préparation puis prêt.
6. **Temps réel** : montrer la notification ou le changement d'état côté staff.
7. **Paiement QR** : générer le QR, scanner, confirmer le paiement.
8. **Table libérée** : vérifier que la commande devient payée et la table disponible.
9. **Avis client** : laisser un commentaire sans étoiles.
10. **Analyse sentiment** : montrer le label et son impact sur les stats.

Comptes démo

Mot de passe commun : `password123`. Comptes : `gerant_test`, `serveur_test`, `cuisinier_test`, `client_test`.

5 Fiche IA / NLP

5.1 Ce que fait l'IA dans Tastify

L'IA ne remplace pas le gérant. Elle l'aide à lire rapidement les retours clients. Quand un client écrit un avis, le texte est analysé pour produire :

- un label : POSITIF, NEUTRE ou NEGATIF ;
- un score de confiance ;
- le modèle utilisé ;
- un score métier exploitable par le dashboard et les recommandations.

Pipeline de sentiment

Avis client → API `/api/avis/` → sauvegarde rapide → tâche Celery → Hugging Face ou fallback local → stockage du label → dashboard et recommandations.

5.2 Modèles utilisés

Modèle / approche	Utilisation
nlpTown/bert-base-multilingual-uncased-sentiment	Modèle principal pour les avis non arabes. Adapté aux avis courts et multilingues.
moossaKam/MARBERT-sentiment	Modèle spécialisé pour les textes arabes.
Fallback lexical multilingue	Secours local si l'API Hugging Face est indisponible ou non configurée.

Réponse simple : pourquoi BERT ?

BERT comprend le contexte d'une phrase grâce aux mécanismes d'attention. C'est plus robuste qu'une simple liste de mots, surtout quand l'avis est court ou ambigu. Dans Tastify, nous utilisons un modèle déjà entraîné pour les avis, ce qui correspond bien au domaine restaurant.

Réponse simple : pourquoi un fallback ?

Parce qu'une fonctionnalité utile ne doit pas tomber complètement si Internet ou l'API externe ne répond pas. Le fallback est moins précis, mais il garantit une continuité de service pendant la démo et dans un usage dégradé.

5.3 Chiffres IA à connaître

Élément	Résultat à citer
Corpus de validation	15 avis réalistes en français et darija latinisée.
Fallback local	Accuracy indicative de 0,53 ; utile comme secours mais limité.
Pipeline distant Hugging Face	Accuracy indicative de 0,93 ; macro-moyenne F1 autour de 0,92.
Erreur typique	Avis mixtes ou ironiques, par exemple positif sur le cadre mais négatif sur la nourriture.

6 Questions jury : réponses prêtes

6.1 Architecture et choix techniques

Question possible	Réponse courte
Pourquoi deux applications front-end ?	Parce que le client et le staff n'ont pas les mêmes besoins. Le client doit avoir une interface simple. Le staff a besoin d'un outil métier plus dense avec salle, KDS, stock et statistiques.
Pourquoi Django REST ?	Django apporte une base solide : ORM, sécurité, authentification, admin, tests et structuration par applications. DRF facilite la création d'API REST maintenables.
Pourquoi React ?	React permet de construire des interfaces dynamiques et réactives, adaptées au panier client, au plan de salle et au KDS.
Pourquoi Redis et Celery ?	Redis sert au temps réel et au broker. Celery déporte les traitements longs, comme l'analyse de sentiment, pour ne pas bloquer l'utilisateur.
Pourquoi WebSocket ?	Le KDS et les notifications staff doivent se mettre à jour sans rafraîchir la page. WebSocket est adapté à ce besoin temps réel.

6.2 Modules fonctionnels à défendre

Module	Ce qu'il faut dire
Menu	Le menu est organisé par catégories et plats. Le staff peut gérer disponibilité, prix, images et composition. Côté client, seuls les plats disponibles doivent être commandables.
Salle	Le plan de salle donne au serveur une vision immédiate des tables libres, occupées ou en paiement. C'est le point d'entrée naturel pour créer une commande sur place.
Commandes	La commande garde les lignes, les quantités et les prix au moment de l'achat. Cela évite qu'une modification future du prix d'un plat change une ancienne commande.
KDS cuisine	Le Kitchen Display System permet au cuisinier de voir les plats à préparer et de changer leur statut. Le serveur est informé en temps réel.
Réservations	Le client peut réserver, et le staff peut suivre les demandes. Le système évite les conflits de disponibilité selon les règles prévues.
Paielements	Le paiement est simulé mais suit une logique métier complète : montant, commande, token QR, confirmation, statut payé et libération de table.
Stock	Les ingrédients, seuils et recettes relient le menu à la réalité cuisine. Un plat dépend de ses ingrédients, pas seulement d'un prix et d'une image.
Avis	Les avis clients sont stockés puis analysés. Le gérant peut voir les retours défavorables et suivre la satisfaction globale.
Fidélité	Le programme fidélité permet de suivre points, récompenses et transactions afin d'encourager le retour client.
Analytics	Le dashboard synthétise les données : revenus, commandes, plats populaires, stock et sentiment client.

6.3 Ce que le jury teste vraiment

Lire entre les lignes

Quand le jury pose une question technique, il ne cherche pas toujours une réponse très longue. Il veut vérifier quatre choses : est-ce que vous comprenez votre architecture, est-ce que vous savez justifier vos choix, est-ce que vous connaissez les limites, et est-ce que vous savez proposer une amélioration réaliste.

S'ils demandent...	Ils veulent vérifier...
Pourquoi cette technologie ?	Que le choix n'est pas aléatoire et répond au besoin métier.
Est-ce sécurisé ?	Que la sécurité est côté backend, pas seulement dans l'interface.
Est-ce vraiment de l'IA ?	Que vous comprenez la différence entre intégrer un modèle et entraîner un modèle.
Est-ce prêt pour production ?	Que vous distinguez prototype PFA, démo robuste et production réelle.
Comment avez-vous testé ?	Que le projet n'est pas seulement visuel, mais validé par des tests.
Quelle est la limite ?	Que vous êtes honnête et capable de prioriser les évolutions.

6.4 Sécurité

Question possible	Réponse courte
Comment gérez-vous les rôles ?	Chaque utilisateur a un rôle : gérant, serveur, cuisinier ou client. Le frontend masque les menus non autorisés, mais la vraie sécurité est appliquée côté API avec les permissions.
Le QR de paiement est-il sécurisé ?	Le QR contient un token signé lié à une commande et à une table. Il ne donne pas un accès général au système.
Le paiement est-il réel ?	Non, c'est une simulation PFA. Mais la structure est prête pour intégrer un prestataire réel comme Stripe, CMI ou PayPal.
Où sont les secrets ?	Les secrets doivent rester dans les variables d'environnement. Ils ne doivent pas être hardcodés dans le code.

6.5 Données et modèle relationnel

Entité	Rôle à expliquer simplement
Utilisateur	Compte avec rôle : client, serveur, cuisinier ou gérant.
Plat / Catégorie	Structure de la carte et disponibilité commerciale.
Ingredient / Recette	Lien entre ce qu'on vend et ce qu'on consomme en stock.
Table	Support de la salle, des commandes sur place et du paiement QR.
Commande / Ligne	Historique de ce qui est commandé, préparé, servi et payé.
Paieiment	Trace de l'encaissement simulé et du statut financier de la commande.
Avis / AnalyseSentiment	Texte client, polarité, score et exploitation analytique.
LoyaltyProfile	Suivi des points et récompenses du client.

Phrase utile sur le modèle de données

Le modèle de données suit les objets réels du restaurant : client, table, plat, commande, paiement, ingrédient et avis. Cela rend l'application plus simple à expliquer, parce que chaque table importante correspond à un concept métier concret.

6.6 Métier restaurant

Question possible	Réponse courte
Quand le stock diminue-t-il ?	Quand un plat part en préparation, pas quand il est juste ajouté au panier. Cela correspond mieux au flux réel en cuisine.
Que se passe-t-il si un ingrédient manque ?	Les recettes relient les plats aux ingrédients. Si un ingrédient est insuffisant, le plat peut devenir indisponible pour éviter de vendre un plat impossible à préparer.
Pourquoi un KDS ?	Le KDS remplace les bons papier en cuisine et réduit les oublis. Le cuisinier voit les commandes et change leur statut en temps réel.
À quoi sert la fidélité ?	Elle améliore la relation client : points, transactions, récompenses et suivi du compte.

6.7 IA : questions plus poussées

Question possible	Réponse courte
Quelle est la différence IA, ML et Deep Learning ?	L'IA est le domaine général. Le Machine Learning apprend à partir de données. Le Deep Learning est une sous-famille du ML basée sur des réseaux de neurones profonds. BERT appartient à cette famille.
Qu'est-ce que le NLP ?	Le NLP est le traitement automatique du langage naturel. Ici, il sert à classifier un commentaire client en sentiment positif, neutre ou négatif.
Pourquoi ne pas utiliser seulement les étoiles ?	Les étoiles donnent une note explicite, mais le texte contient plus d'information : lenteur, goût, température, service. L'analyse de sentiment valorise le commentaire.
Comment convertissez-vous les sorties ?	Les sorties comme 5 stars, POSITIVE ou LABEL_2 sont normalisées en POSITIF. Les sorties négatives deviennent NEGATIF ; les cas moyens deviennent NEUTRE.
Pourquoi l'arabe a un modèle séparé ?	Les avis peuvent contenir de l'arabe ou du dialecte. MARBERT est plus adapté à ces textes qu'un modèle général principalement orienté langues européennes.
Comment améliorer l'IA ?	Collecter un corpus réel d'avis Tastify, l'annoter, comparer plusieurs modèles, mieux détecter les langues et combiner texte + note.

6.8 Tests et qualité

Question possible	Réponse courte
Comment avez-vous validé le projet ?	Avec des tests backend pytest, des tests unitaires frontend Vitest, des scénarios end-to-end Playwright et une validation locale complète.
Quels chiffres retenir ?	346 tests backend réussis, 93% de couverture, 121 scénarios Playwright back-office et 81 scénarios Playwright client.
Que couvrent les tests ?	Authentification, permissions, commandes, KDS, paiements, stock, réservations, avis, sentiment, WebSockets et parcours client/staff.
Pourquoi les tests E2E ?	Ils vérifient les vrais parcours utilisateur, pas seulement les fonctions isolées. C'est important pour une application métier complète.

6.9 Déploiement et environnement

Question possible	Réponse courte
Comment lancez-vous la démo ?	Avec Docker Desktop puis <code>start-demo-local.bat</code> . Le script prépare les services, les données de démo et les URLs.
Quels services tournent ?	Backend Django, base MySQL, Redis, Celery worker, Celery beat et les deux frontends.
Pourquoi pas directement en cloud ?	Le PFA vise d'abord une solution fonctionnelle et démontrable. Le cloud est une perspective avec HTTPS, monitoring, sauvegardes et secrets managés.
Que faire si Hugging Face n'est pas configuré ?	La démo continue avec le fallback local. C'est prévu pour la robustesse.

7 Plan de parole conseillé

Temps	Contenu
0 :00–1 :00	Saluer, présenter Tastify, poser la problématique et annoncer la valeur ajoutée.
1 :00–2 :30	Expliquer les acteurs : client, gérant, serveur, cuisinier. Montrer que le système couvre tout le restaurant.
2 :30–4 :00	Présenter l'architecture sans trop détailler : deux frontends, backend commun, base, Redis, Celery, Docker.
4 :00–11 :00	Faire la démo : portail client, back-office, salle, KDS, paiement QR, avis et sentiment.
11 :00–12 :30	Présenter les tests et chiffres de validation.
12 :30–14 :00	Présenter limites et perspectives.
14 :00–15 :00	Conclusion claire, puis ouverture aux questions.

Règle d'or pendant la soutenance

Ne pas réciter tout le rapport. Pour chaque écran, dire : à quoi ça sert, quel acteur l'utilise, quelle donnée est créée ou mise à jour, et quel choix technique se cache derrière.

8 Transitions orales utiles

- Après la problématique : “Pour répondre à cela, nous avons séparé le système en deux expériences : client et staff.”
- Avant l'architecture : “Derrière ces interfaces, le backend reste commun pour garder une seule source de vérité.”
- Avant le KDS : “Ici, l'intérêt n'est pas seulement d'enregistrer une commande, mais de la transmettre immédiatement à la cuisine.”
- Avant le paiement : “Nous avons simulé le paiement pour valider tout le flux métier sans dépendre d'une banque.”
- Avant l'IA : “La partie IA intervient après l'expérience client, quand le commentaire devient une donnée exploitable.”
- Avant les tests : “Pour éviter une simple démonstration visuelle, nous avons validé les parcours critiques par des tests.”
- Avant les limites : “Le projet est fonctionnel pour le PFA, mais nous avons identifié ce qu'il faudrait renforcer pour une production réelle.”

9 Questions difficiles : rester honnête

Si le jury dit : “Votre IA est-elle vraiment entraînée par vous ?”

Réponse : Non, nous n'avons pas entraîné un grand modèle depuis zéro. Nous avons intégré des modèles pré-entraînés via Hugging Face et nous avons adapté leurs sorties au besoin métier de Tastify. C'est un choix réaliste pour un PFA : exploiter un modèle fiable, l'intégrer proprement, prévoir un fallback et mesurer son comportement sur un corpus de validation.

Si le jury dit : “Le fallback local est faible”

Réponse : Oui, et nous le présentons comme une solution de secours, pas comme le moteur principal. Il assure la continuité si l'API externe n'est pas disponible. Une perspective serait de le remplacer par un modèle local léger entraîné sur un corpus d'avis annotés.

Si le jury dit : “Le paiement n'est pas réel”

Réponse : C'est volontaire pour le périmètre PFA. Nous avons simulé le paiement pour démontrer le flux métier complet : commande, montant, QR, confirmation, état payé et libération de table. L'intégration d'un vrai prestataire serait une évolution naturelle.

Si la démo bug

Réponse calme : Je vais expliquer le flux prévu et montrer la partie déjà chargée. L'architecture Docker permet normalement de relancer proprement. Les tests exécutés montrent que les parcours principaux sont validés ; un incident de démo reste un problème d'environnement, pas forcément un problème de conception.

10 Limites et perspectives

Limite actuelle	Perspective naturelle
Paiement simulé	Intégrer Stripe, CMI ou PayPal avec webhooks et rapprochement bancaire.
Corpus IA limité	Construire un corpus réel d'avis Tastify annotés manuellement.
Fallback lexical simple	Ajouter un modèle local léger TF-IDF + classifieur ou un petit transformer local.
Recommandation simple	Ajouter historique client, contraintes de stock et préférences individuelles.
Prévision stock basique	Ajouter une vraie prévision avec historique, saisonnalité et événements.
Tests de charge à finaliser	Construire un scénario Locust avec temps moyen, p95 et taux d'échec.
Production cloud	HTTPS, monitoring, sauvegardes, rotation des secrets et observabilité.

11 Phrases courtes à placer

- Tastify centralise le parcours restaurant.
- Le backend commun garantit la cohérence métier.
- La séparation client/staff respecte les usages.
- Les WebSockets rendent la cuisine réactive.
- Celery évite de bloquer l'utilisateur.
- L'IA transforme les avis en indicateurs.
- Le fallback rend la démo robuste.
- Les permissions sont vérifiées côté API.
- Le QR contient un token signé.
- Le paiement est simulé mais le flux métier est complet.
- Les tests couvrent les parcours critiques.
- Les limites sont identifiées et transformées en perspectives.

12 Checklist de veille et jour J

Avant de dormir

- Relire le pitch, les chiffres et les questions difficiles.
- Répéter la démo en parlant à voix haute.
- Préparer les onglets : client, gérant, serveur, cuisinier.
- Vérifier Docker Desktop et le téléphone sur le même Wi-Fi si QR prévu.

Le jour J

- Lancer `start-demo-local.bat` au moins 10 minutes avant.
- Garder les comptes démo prêts.
- Désactiver le VPN si le téléphone ne rejoint pas l'IP du PC.
- Si le QR pointe vers `localhost`, relancer le script de démo.
- Répondre simplement : problème, choix, preuve, limite, perspective.

13 Mini conclusion prête à dire

Conclusion orale

Pour conclure, Tastify répond à un besoin concret de digitalisation d'un restaurant. Le projet couvre les parcours client et staff, intègre le temps réel, la gestion du stock, le paiement simulé, la fidélité et une couche IA pour analyser les avis. Les tests montrent une base solide, et les limites restantes ouvrent vers une version production plus complète avec paiement réel, monitoring, corpus IA enrichi et recommandations plus avancées.